

“Lost and Found Item Tracker”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KLH BACHUPALLY

Project Theme: Eye of Odin – Campus Lost-and-Found Information Management

Academic Year: 2026–2027 | Cluster: 3 | Project Type: Trimester-end

By

VINAY BANDLA

2520030437

HANSIKA REDDY GUNAPATI

2520030237

TEJA HASINI REDDY

2520030048

Under the Esteemed Guidance of

Dr. S. Spandana

Dr. Y. Subbarayudu

Assistant Professors

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

KLH BACHUPALLY (Deemed to be University)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

We hereby declare that the project report entitled **“Lost and Found Item Tracker”** is a record of the work carried out by **Vinay Bandla (2520030437)**, **Hansika Reddy Gunapati (2520030237)**, and **Teja Hasini Reddy (2520030048)**, students of **B.Tech**, Department of Computer Science and Engineering, KLH Bachupally, during the academic year 2026–2027.

The report presents the project problem, requirements, architecture, design, implementation approach, testing strategy, deployment considerations, limitations, and future enhancement plan for the Eye of Odin application.

The documentation distinguishes implemented functionality from proposed functionality. In particular, the current project foundation is described around FastAPI, Pydantic, SQLAlchemy, SQLite, image-upload validation, report retrieval, filtering, updating, and automated testing. Computer-vision matching, notifications, and a complete administrative interface are treated as future extensions unless separately demonstrated by project evidence.

We affirm that the technical descriptions in this report are intended to represent the project repository and the work performed by the team. Any external documentation or framework material used to understand the implementation is identified in the references section.

Student Signatures

Vinay Bandla – 2520030437

Hansika Reddy Gunapati – 2520030237

Teja Hasini Reddy – 2520030048

KLH BACHUPALLY (Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the Trimester-end project report entitled **“Lost and Found Item Tracker”** is a bonafide work carried out by **Vinay Bandla (2520030437)**, **Hansika Reddy Gunapati (2520030237)**, and **Teja Hasini Reddy (2520030048)**, students of **B.Tech**, Department of Computer Science and Engineering, KLH Bachupally, during the academic year 2026–2027.

The project was developed under the guidance of Dr. S. Spandana and Dr. Y. Subbarayudu, Assistant Professors, Department of Computer Science and Engineering. The report is submitted as part of the academic requirements for the Trimester-end project.

The work described in the report covers the problem definition, requirements analysis, technology selection, architecture, design, implementation, testing, deployment considerations, limitations, and future enhancement roadmap of the Eye of Odin project.

Date: 03-10-2026

Place: KLH Bachupally

Signature of the Guide

Dr. S. Spandana / Dr. Y. Subbarayudu

Signature of the Course Coordinator

Ramatenki Sateesh Kumar

Signature of the HOD

Dr. Venkateswara Rao

ACKNOWLEDGEMENT

We express our sincere gratitude to K LH Bachupally and the Department of Computer Science and Engineering for providing the academic environment, project framework, laboratory facilities, and review opportunities required for completing this work.

We thank our guides, Dr. S. Spandana and Dr. Y. Subbarayudu, for their guidance, technical suggestions, reviews, and encouragement. Their feedback helped us organize the project around a clear problem statement and a maintainable software architecture rather than treating the application as only a collection of screens or API calls.

We are grateful to our Course Coordinator, Ramatenki Sateesh Kumar, and our Head of the Department, Dr. Venkateswara Rao, for their academic support and for providing a structured environment in which the project could be planned, reviewed, tested, and documented.

We also acknowledge the contribution of classmates, faculty members, and others who provided observations about campus lost-and-found workflows. These discussions helped us recognize the importance of consistent categories, locations, descriptions, identifiers, and evidence when handling reports.

Finally, we thank our families and friends for their encouragement throughout the project period. The completion of this report reflects a combination of technical practice, documentation discipline, review feedback, and teamwork.

Student Signatures

VINAY BANDLA – 2520030437

HANSIKA REDDY GUNAPATI – 2520030237

TEJA HASINI REDDY – 2520030048

Abstract

The Lost and Found Item Tracker, developed under the project name “Eye of Odin”, is a campus-oriented system for organizing information about lost and found belongings. Informal reporting through messaging groups, paper registers, announcements, or word of mouth can make records difficult to search, preserve, compare, and update. The project addresses this information-management problem by providing a structured backend foundation for creating, retrieving, filtering, and updating reports.

The documented implementation uses FastAPI as the API layer, Pydantic for request and response validation, SQLAlchemy for database interaction, and SQLite for persistent storage. The system also includes image-upload handling with file-content validation. Supported categories and campus locations help reduce inconsistent terminology and make reports easier to filter.

The project is designed with separation of concerns in mind. API routes handle communication, schemas define validated data structures, models represent persistent entities, and database utilities support storage operations. Automated tests using pytest and FastAPI TestClient provide a repeatable method for checking important endpoint behaviors.

The present implementation is intentionally distinguished from the future roadmap. Image-based matching using CLIP embeddings and cosine similarity is documented as a planned enhancement rather than presented as an already completed capability. Future work can also include notifications, an administrative dashboard, stronger authentication, scalable storage, and a richer user interface.

Overall, the project demonstrates how a relatively simple campus problem can be converted into a structured software engineering exercise involving requirements analysis, API design, validation, persistence, file handling, testing, deployment planning, and future extensibility.

Index

S. No.	Chapters	Topics	Page.no
		Declaration	2
		Certificate	3
		Acknowledgement	4
		Abstract	5
		List of Tables and Figures	9
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	10-22
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks 2.4 Functional Requirements 2.5 Non-Functional Requirements	23-29
3	Technology Stack	3.1 Backend Technology 3.2 Pydantic Validation 3.3 SQLAlchemy ORM 3.4 SQLite 3.5 Uvicorn 3.6 Image Handling 3.7 Testing Stack 3.8 Version Control 3.9 Planned Computer Vision 3.10 Technology Stack Summary	30-42
4	System Architecture	4.1 High-Level Architecture 4.2 Architecture Layers 4.3 Data Flow 4.4 Future Matching Architecture	43-47
5	Design	5.1 Design and Database Schema	48-49
6	Implementation	6.1 Implementation Overview 6.2 Application Entry Point 6.3 Database Module 6.4 Model Layer 6.5 Schema Layer 6.6 Report Creation 6.7 Retrieval, Listing and Filtering 6.8 Update Operations 6.9 Image Upload 6.10 Testing and Implementation Quality	50-61

S. No.	Chapters	Topics	Page.no
7	Features	7.1 Lost Item Reporting 7.2 Found Item Reporting 7.3 Report Search and Filtering 7.4 Report Identification 7.5 Image Support 7.6 Validation and Data Quality 7.7 Administrative Extension 7.8 Eye of Odin Concept	62-70
8	Testing	8.1 Testing Strategy 8.2 Create Report Tests 8.3 Retrieval Tests 8.4 Filtering Tests 8.5 Update Tests 8.6 Image Tests 8.7 Integration Testing 8.8 Test Cases and Expected Results 8.9 Testing Tools and Reporting	71-80
9	Deployment	9.1 Local Deployment 9.2 Environment Setup 9.3 Configuration 9.4 API Verification 9.5 Production Considerations 9.6 Image Storage Considerations 9.7 Security Considerations 9.8 Backup and Recovery 9.9 Deployment Workflow	81-91
10	Challenges & Limitations	10.1 Challenge – Fragmented Reporting 10.2 Challenge – Inconsistent Terminology 10.3 Challenge – User-Uploaded Files 10.4 Challenge – Matching Ambiguity 10.5 Challenge – Prototype Scale 10.6 Challenge – Frontend Dependency 10.7 Current Limitations 10.8 Solutions Applied 10.9 Known Risks and Mitigation	92-102
11	Future Enhancements	11.1 CLIP-Based Image Representation 11.2 Cosine Similarity 11.3 Candidate Matching 11.4 Notifications 11.5 Admin Dashboard and Analytics 11.6 Scalable Architecture	103-109

S. No.	Chapters	Topics	Page.no
12	Conclusion	12.1 Summary 12.2 Achievement – API Foundation 12.3 Achievement – Data Management 12.4 Achievement – Image Handling 12.5 Achievement – Testing 12.6 Skills Learned 12.7 Academic Significance 12.8 Future Direction 12.9 Conclusion	110-120
13	References	13.1 References – Project and Framework Documentation 13.2 References – Future Computer Vision and General Tools	121-123
14	Appendices	14.1 Appendix – Project and Team Details 14.2 Appendix – Guide and Academic Details 14.3 Appendix – Recommended Project Screenshots 14.4 Appendix – Suggested API Evidence 14.5 Appendix – User Guide 14.6 Appendix – Review Forms and Future Matching Pseudocode	124-130

List of Tables and Figures

Reference	Title
Table 2.1	Hardware Requirements
Table 2.2	Software Requirements
Table 2.3	Functional Requirements
Table 2.4	Non-Functional Requirements
Table 3.1	Technology Stack Summary
Table 4.1	Architecture Layers and Responsibilities
Table 5.1	Core Report Fields
Table 6.1	Implementation Modules
Table 7.1	Supported Item Categories
Table 7.2	Supported Campus Locations
Table 8.1	Test Case Matrix
Table 9.1	Deployment Checklist
Table 10.1	Challenge and Mitigation Matrix
Table 11.1	Future Enhancement Roadmap
Table 12.1	Project Achievements
Figure 4.1	High-Level Architecture
Figure 4.2	Report Data Flow
Figure 5.1	Report Lifecycle
Figure 6.1	API Interaction Sequence
Figure 8.1	Testing Workflow
Figure 9.1	Deployment Workflow
Appendix A	Project Team and Academic Details
Appendix B	Recommended Screenshots and API Evidence
Appendix C	User Guide and Future Matching Pseudocode

The listed figures and tables are intended as report artifacts. Where a screenshot or diagram is required for final academic submission, it should be replaced by evidence captured from the actual implementation.

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

The engineering evidence for this area should be observable. For the campus context, common loss scenarios, and why a dedicated digital record is needed, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the campus context, common loss scenarios, and why a dedicated digital record is needed. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the campus context, common loss scenarios, and why a dedicated digital record is needed. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the campus context, common loss scenarios, and why a dedicated digital record is needed. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the campus context, common loss scenarios, and why a dedicated digital record is needed. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Review lens: verify that the described behavior can be tested with a concrete request, response, record, or artifact.

1.1 Background of the Project – Need for Digital Records

A manual register provides persistence but usually requires physical access and manual searching. Informal messages provide fast communication but are easily buried by newer messages. A centralized API-backed system combines persistence with structured retrieval.

The project therefore focuses first on reliable record management. Intelligent matching is positioned as a later stage rather than a requirement for the basic reporting workflow.

There is also an important scope boundary around the transition from informal messages and registers to persistent, searchable records. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the transition from informal messages and registers to persistent, searchable records. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the transition from informal messages and registers to persistent, searchable records. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the transition from informal messages and registers to persistent, searchable records. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the transition from informal messages and registers to persistent, searchable records, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates the campus context, common loss scenarios, and why a dedicated digital record is needed. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

1.2 Problem Statement

The central problem is the absence of a consistent, searchable mechanism for managing campus lost-and-found reports. A student who loses an item may not know where a corresponding found report has been recorded. Conversely, a person who finds an item may not have a reliable way to identify the owner.

The system addresses this gap by providing common data structures, controlled categories and locations, API operations, persistent storage, and image handling. These capabilities establish a common information layer for future user interfaces and matching services.

The central point of this section is the central information-management gap between people who lose items and people who find them. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the central information-management gap between people who lose items and people who find them. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the central information-management gap between people who lose items and people who find them. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the central information-management gap between people who lose items and people who find them, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the central information-management gap between people who lose items and people who find them. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates the transition from informal messages and registers to persistent, searchable records. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

1.2 Problem Statement – Information Fragmentation

Fragmentation creates several practical issues: duplicate reporting, inconsistent terminology, missing descriptions, and difficulty following the lifecycle of an item. The same physical item may be discussed in several unrelated channels.

A centralized report identifier gives each record a stable reference. Filtering also becomes possible when category, location, or status is represented consistently.

From an implementation perspective, fragmented communication, duplicate reports, inconsistent wording, and the value of stable identifiers. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain fragmented communication, duplicate reports, inconsistent wording, and the value of stable identifiers. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For fragmented communication, duplicate reports, inconsistent wording, and the value of stable identifiers, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around fragmented communication, duplicate reports, inconsistent wording, and the value of stable identifiers. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is fragmented communication, duplicate reports, inconsistent wording, and the value of stable identifiers. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates the central information-management gap between people who lose items and people who find them. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

1.3 Scope of the Project

The scope includes the backend management of lost and found reports, structured validation, persistence through SQLite and SQLAlchemy, report retrieval and filtering, updates, and image upload validation.

The project is also designed as a foundation for a future computer-vision module. The documented roadmap identifies CLIP embeddings and cosine similarity for image-based candidate matching.

A representative campus situation helps explain the project boundary: report creation, retrieval, filtering, updates, persistence, and image validation. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the project boundary: report creation, retrieval, filtering, updates, persistence, and image validation, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the project boundary: report creation, retrieval, filtering, updates, persistence, and image validation. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the project boundary: report creation, retrieval, filtering, updates, persistence, and image validation. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the project boundary: report creation, retrieval, filtering, updates, persistence, and image validation. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates fragmented communication, duplicate reports, inconsistent wording, and the value of stable identifiers. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

1.3 Scope – Included Functions

The included functions can be grouped into reporting, retrieval, update, persistence, and image handling. The system defines supported item categories and campus locations so that reports can be represented consistently.

The scope is deliberately modular. A frontend, authentication layer, notifications, and intelligent matching can consume or extend the API without changing the basic idea of a structured report.

The engineering evidence for this area should be observable. For the functions deliberately included in the present backend and the interfaces that can extend it later, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the functions deliberately included in the present backend and the interfaces that can extend it later. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the functions deliberately included in the present backend and the interfaces that can extend it later. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the functions deliberately included in the present backend and the interfaces that can extend it later. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the functions deliberately included in the present backend and the interfaces that can extend it later. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates the project boundary: report creation, retrieval, filtering, updates, persistence, and image validation. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

1.4 Objectives

The primary objective is to develop a reliable campus lost-and-found tracking foundation. The project aims to make reports easier to create, store, retrieve, filter, and update.

A second objective is extensibility. By separating API routes, schemas, models, database access, and image handling, the project can support future modules without turning the core application into a single monolithic component.

There is also an important scope boundary around project objectives covering reliability, usability, maintainability, structured data, and extensibility. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is project objectives covering reliability, usability, maintainability, structured data, and extensibility. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, project objectives covering reliability, usability, maintainability, structured data, and extensibility. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain project objectives covering reliability, usability, maintainability, structured data, and extensibility. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For project objectives covering reliability, usability, maintainability, structured data, and extensibility, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates the functions deliberately included in the present backend and the interfaces that can extend it later. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

1.4 Objectives – Technical Objectives

The technical objectives include implementing REST-style operations, validating incoming data, persisting records, handling uploaded images safely, and testing endpoint behavior.

The architecture should remain understandable to developers and suitable for incremental enhancement. The planned matching workflow should be able to operate as an additional service or module.

The central point of this section is technical objectives around REST-style APIs, validation, persistence, file handling, and automated tests. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, technical objectives around REST-style APIs, validation, persistence, file handling, and automated tests. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain technical objectives around REST-style APIs, validation, persistence, file handling, and automated tests. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For technical objectives around REST-style APIs, validation, persistence, file handling, and automated tests, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around technical objectives around REST-style APIs, validation, persistence, file handling, and automated tests. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates project objectives covering reliability, usability, maintainability, structured data, and extensibility. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

1.5 Importance of the Application

The application is important because a lost-and-found system is useful only when people can discover relevant information. Structured records make reports searchable and preserve them beyond a temporary announcement.

From an academic perspective, the project demonstrates practical use of web APIs, database modeling, validation, file handling, automated testing, and modular software design.

From an implementation perspective, the practical value of searchable records and the academic value of integrating several software concepts. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the practical value of searchable records and the academic value of integrating several software concepts. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the practical value of searchable records and the academic value of integrating several software concepts, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the practical value of searchable records and the academic value of integrating several software concepts. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the practical value of searchable records and the academic value of integrating several software concepts. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates technical objectives around REST-style APIs, validation, persistence, file handling, and automated tests. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

1.6 Target Users / Audience

The primary audience includes students and staff who lose or find belongings on campus. A future administrator interface can support authorized staff responsible for reviewing reports and resolving claims.

The system can also serve as a foundation for future developers who want to add authentication, notifications, dashboards, or visual matching.

A representative campus situation helps explain students, staff, campus support personnel, and future administrators as distinct user groups. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For students, staff, campus support personnel, and future administrators as distinct user groups, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around students, staff, campus support personnel, and future administrators as distinct user groups. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is students, staff, campus support personnel, and future administrators as distinct user groups. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, students, staff, campus support personnel, and future administrators as distinct user groups. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates the practical value of searchable records and the academic value of integrating several software concepts. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

1.6 Expected Benefits

The expected benefit is a more organized information flow. Users can work with consistent report records instead of relying entirely on informal communication.

The project also provides a clear technical foundation for later intelligence. Once reports and images are stored consistently, similarity-based candidate discovery becomes a feasible extension.

The engineering evidence for this area should be observable. For expected benefits such as discoverability, consistent records, traceability, and a foundation for future matching, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around expected benefits such as discoverability, consistent records, traceability, and a foundation for future matching. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is expected benefits such as discoverability, consistent records, traceability, and a foundation for future matching. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, expected benefits such as discoverability, consistent records, traceability, and a foundation for future matching. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain expected benefits such as discoverability, consistent records, traceability, and a foundation for future matching. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates students, staff, campus support personnel, and future administrators as distinct user groups. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

There is also an important scope boundary around baseline hardware requirements for Python development, database work, browser testing, and local API execution. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is baseline hardware requirements for Python development, database work, browser testing, and local API execution. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, baseline hardware requirements for Python development, database work, browser testing, and local API execution. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain baseline hardware requirements for Python development, database work, browser testing, and local API execution. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For baseline hardware requirements for Python development, database work, browser testing, and local API execution, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates expected benefits such as discoverability, consistent records, traceability, and a foundation for future matching. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

2.1 Hardware – Development and Client Context

The development machine should provide enough storage for the source repository, Python environment, dependencies, test data, database file, and uploaded development images. A modern browser is useful for API documentation and frontend testing.

End users need only a device capable of accessing the eventual web interface. The system does not require specialized hardware for basic report management.

The central point of this section is development-machine and client considerations including memory, storage, browser access, and network assumptions. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, development-machine and client considerations including memory, storage, browser access, and network assumptions. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain development-machine and client considerations including memory, storage, browser access, and network assumptions. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For development-machine and client considerations including memory, storage, browser access, and network assumptions, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented

roadmap.

There is also an important scope boundary around development-machine and client considerations including memory, storage, browser access, and network assumptions. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates baseline hardware requirements for Python development, database work, browser testing, and local API execution. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

2.2 Software Requirements

The core software stack consists of Python, FastAPI, Uvicorn, Pydantic, SQLAlchemy, SQLite, Pytest, and FastAPI TestClient. Git and GitHub support version control and collaboration.

A code editor or IDE is required for development. The application can be run locally before being prepared for a production server.

From an implementation perspective, the software environment required for Python, the application server, database layer, testing, and source control. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the software environment required for Python, the application server, database layer, testing, and source control. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the software environment required for Python, the application server, database layer, testing, and source control, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented

roadmap.

There is also an important scope boundary around the software environment required for Python, the application server, database layer, testing, and source control. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the software environment required for Python, the application server, database layer, testing, and source control. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates development-machine and client considerations including memory, storage, browser access, and network assumptions. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

2.3 Development Tools and Frameworks

FastAPI handles HTTP routing and API requests. Pydantic validates structured data. SQLAlchemy provides the persistence abstraction and SQLite provides the database. Pytest and TestClient support automated API testing.

The separation of these tools into clear responsibilities supports maintainability.

A representative campus situation helps explain the role of the development toolkit in building, running, testing, documenting, and maintaining the application. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the role of the development toolkit in building, running, testing, documenting, and maintaining the application, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the role of the development toolkit in building, running, testing, documenting, and maintaining the application. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the role of the development toolkit in building, running, testing, documenting, and maintaining the application. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the role of the development toolkit in building, running, testing, documenting, and maintaining the application. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates the software environment required for Python, the application server, database layer, testing, and source control. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

2.4 Functional Requirements

The system should create lost/found reports, list reports, retrieve an individual report, filter report information where supported, update records, and process item images.

Invalid input should be rejected instead of being silently stored. The application should provide predictable responses for successful operations and errors.

The engineering evidence for this area should be observable. For functional requirements expressed as concrete actions the system must support for a lost-and-found workflow, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around functional requirements expressed as concrete actions the system must support for a lost-and-found workflow. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is functional requirements expressed as concrete actions the system must support for a lost-and-found workflow. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, functional requirements expressed as concrete actions the system must support for a lost-and-found workflow. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain functional requirements expressed as concrete actions the system must support for a lost-and-found workflow. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates the role of the development toolkit in building, running, testing, documenting, and maintaining the application. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

2.5 Non-Functional Requirements

The project should be usable, maintainable, reliable, and extensible. Input validation should protect data quality, while modular code should make future changes easier.

For production, security, authentication, logging, backups, monitoring, and scalable storage should be added as required.

There is also an important scope boundary around quality requirements covering correctness, reliability, maintainability, validation, security awareness, and responsiveness. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is quality requirements covering correctness, reliability, maintainability, validation, security awareness, and responsiveness. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, quality requirements covering correctness, reliability, maintainability, validation, security awareness, and responsiveness. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain quality requirements covering correctness, reliability, maintainability, validation, security awareness, and responsiveness. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For quality requirements covering correctness, reliability, maintainability, validation, security awareness, and responsiveness, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates functional requirements expressed as concrete actions the system must support for a lost-and-found workflow. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -3 TECHNOLOGY STACK

3.1 Backend Technology – FastAPI

The central point of this section is the overall technology stack and the reason each component has a distinct responsibility. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the overall technology stack and the reason each component has a distinct responsibility. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the overall technology stack and the reason each component has a distinct responsibility. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the overall technology stack and the reason each component has a distinct responsibility, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the overall technology stack and the reason each component has a distinct responsibility. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates quality requirements covering correctness, reliability, maintainability, validation, security awareness, and responsiveness. The artifact may be

a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

3.1 Backend Technology – API Design

The API is organized around report resources. Resource-oriented endpoints provide a consistent way to create, retrieve, list, filter, and update reports.

This design makes the backend consumable by a browser frontend, mobile client, API testing tool, or future administrative dashboard.

From an implementation perspective, FastAPI routes, request handling, response generation, and the API contract exposed to clients. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain FastAPI routes, request handling, response generation, and the API contract exposed to clients. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For FastAPI routes, request handling, response generation, and the API contract exposed to clients, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around FastAPI routes, request handling, response generation, and the API contract exposed to clients. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is FastAPI routes, request handling, response generation, and the API contract exposed to clients. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore

treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates the overall technology stack and the reason each component has a distinct responsibility. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

3.2 Pydantic Validation

Pydantic schemas define expected request and response structures. Validation helps prevent malformed values from entering the application.

Controlled item categories and locations are especially useful because they reduce variations such as different spellings for the same campus location.

A representative campus situation helps explain Pydantic models as the validation boundary for incoming and outgoing structured data. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For Pydantic models as the validation boundary for incoming and outgoing structured data, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around Pydantic models as the validation boundary for incoming and outgoing structured data. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is Pydantic models as the validation boundary for incoming and outgoing structured data. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future

extension.

From an implementation perspective, Pydantic models as the validation boundary for incoming and outgoing structured data. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates FastAPI routes, request handling, response generation, and the API contract exposed to clients. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

3.3 SQLAlchemy ORM

SQLAlchemy provides the database abstraction used to represent application data as models and interact with the database.

Using an ORM separates persistence logic from endpoint code and provides a clearer structure for future database changes.

The engineering evidence for this area should be observable. For SQLAlchemy as the object-relational layer connecting application objects with persistent database records, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around SQLAlchemy as the object-relational layer connecting application objects with persistent database records. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is SQLAlchemy as the object-relational layer connecting application objects with persistent database records. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields,

predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, SQLAlchemy as the object-relational layer connecting application objects with persistent database records. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain SQLAlchemy as the object-relational layer connecting application objects with persistent database records. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates Pydantic models as the validation boundary for incoming and outgoing structured data. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

3.4 SQLite

SQLite is a lightweight relational database suitable for a compact prototype. It requires no separate database server and can store the project records in a local database file.

A production campus deployment with many concurrent users may later use a server-based relational database if scale requires it.

There is also an important scope boundary around SQLite as a lightweight local database suitable for a student prototype and repeatable development. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is SQLite as a lightweight local database suitable for a student prototype and repeatable development. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and

future extension.

From an implementation perspective, SQLite as a lightweight local database suitable for a student prototype and repeatable development. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain SQLite as a lightweight local database suitable for a student prototype and repeatable development. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For SQLite as a lightweight local database suitable for a student prototype and repeatable development, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates SQLAlchemy as the object-relational layer connecting application objects with persistent database records. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

3.5 Unicorn

Unicorn acts as the ASGI server used to run the FastAPI application. During development, it provides the process that accepts HTTP requests and passes them to the application.

Separating the framework from the server follows the standard ASGI application model.

The central point of this section is Unicorn as the ASGI server used to run and verify the FastAPI application during development. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, Uvicorn as the ASGI server used to run and verify the FastAPI application during development. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain Uvicorn as the ASGI server used to run and verify the FastAPI application during development. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For Uvicorn as the ASGI server used to run and verify the FastAPI application during development, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around Uvicorn as the ASGI server used to run and verify the FastAPI application during development. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates SQLite as a lightweight local database suitable for a student prototype and repeatable development. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

3.6 Image Handling

Images are treated as user-supplied files and therefore require validation. The documented project includes file-content or magic-byte validation so that a filename extension alone is not trusted.

Validated images can then be associated with reports and can later become inputs to a visual matching service.

From an implementation perspective, image upload handling, file-content validation, and the difference between accepting a file and trusting it. The repository separates responsibilities so that a change in one part does not

automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain image upload handling, file-content validation, and the difference between accepting a file and trusting it. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For image upload handling, file-content validation, and the difference between accepting a file and trusting it, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around image upload handling, file-content validation, and the difference between accepting a file and trusting it. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is image upload handling, file-content validation, and the difference between accepting a file and trusting it. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates Uvicorn as the ASGI server used to run and verify the FastAPI application during development. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

3.7 Testing Stack

Pytest is used for automated test execution, while FastAPI TestClient allows API routes to be exercised in a controlled environment.

Testing should cover valid operations, invalid values, missing records, and image upload behavior.

A representative campus situation helps explain automated tests using pytest and FastAPI TestClient to exercise endpoint behavior consistently. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For automated tests using pytest and FastAPI TestClient to exercise endpoint behavior consistently, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around automated tests using pytest and FastAPI TestClient to exercise endpoint behavior consistently. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is automated tests using pytest and FastAPI TestClient to exercise endpoint behavior consistently. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, automated tests using pytest and FastAPI TestClient to exercise endpoint behavior consistently. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates image upload handling, file-content validation, and the difference between accepting a file and trusting it. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

3.8 Version Control

Git and GitHub provide a history of project changes and make collaborative development easier. The supplied repository is the project reference for Eye of Odin.

Version control also provides a reproducible record of implementation changes and documentation updates.

The engineering evidence for this area should be observable. For version control practices for preserving changes, reviewing code history, and collaborating on the project, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around version control practices for preserving changes, reviewing code history, and collaborating on the project. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is version control practices for preserving changes, reviewing code history, and collaborating on the project. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, version control practices for preserving changes, reviewing code history, and collaborating on the project. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain version control practices for preserving changes, reviewing code history, and collaborating on the project. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates automated tests using pytest and FastAPI TestClient to exercise endpoint behavior consistently. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

3.9 Planned Computer Vision

The project roadmap identifies CLIP as a future method for converting images into embeddings. Embeddings provide numerical representations that can be compared.

The roadmap should be described as planned functionality unless a later implementation has been completed and verified.

There is also an important scope boundary around the planned computer-vision direction using CLIP embeddings as a future extension rather than a current claim. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the planned computer-vision direction using CLIP embeddings as a future extension rather than a current claim. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the planned computer-vision direction using CLIP embeddings as a future extension rather than a current claim. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the planned computer-vision direction using CLIP embeddings as a future extension rather than a current claim. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the planned computer-vision direction using CLIP embeddings as a future extension rather than a current claim, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates version control practices for preserving changes, reviewing code history, and collaborating on the project. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

3.10 Technology Stack Summary

The complete documented stack combines FastAPI, Uvicorn, Pydantic, SQLAlchemy, SQLite, Pytest, TestClient, Git, and GitHub. The planned intelligence layer adds CLIP embeddings and cosine similarity.

Together these components provide a clear progression from basic record management to future visual candidate matching.

The central point of this section is how the selected technologies fit together into a coherent, replaceable, and understandable stack. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, how the selected technologies fit together into a coherent, replaceable, and understandable stack. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain how the selected technologies fit together into a coherent, replaceable, and understandable stack. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For how the selected technologies fit together into a coherent, replaceable, and understandable stack, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around how the selected technologies fit together into a coherent, replaceable, and understandable stack. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates the planned computer-vision direction using CLIP embeddings as a future extension rather than a current claim. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture

From an implementation perspective, the system architecture from client requests through routes, validation, database operations, and responses. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the system architecture from client requests through routes, validation, database operations, and responses. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the system architecture from client requests through routes, validation, database operations, and responses, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the system architecture from client requests through routes, validation, database operations, and responses. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the system architecture from client requests through routes, validation, database operations, and responses. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

4.1 High-Level Architecture (continued)

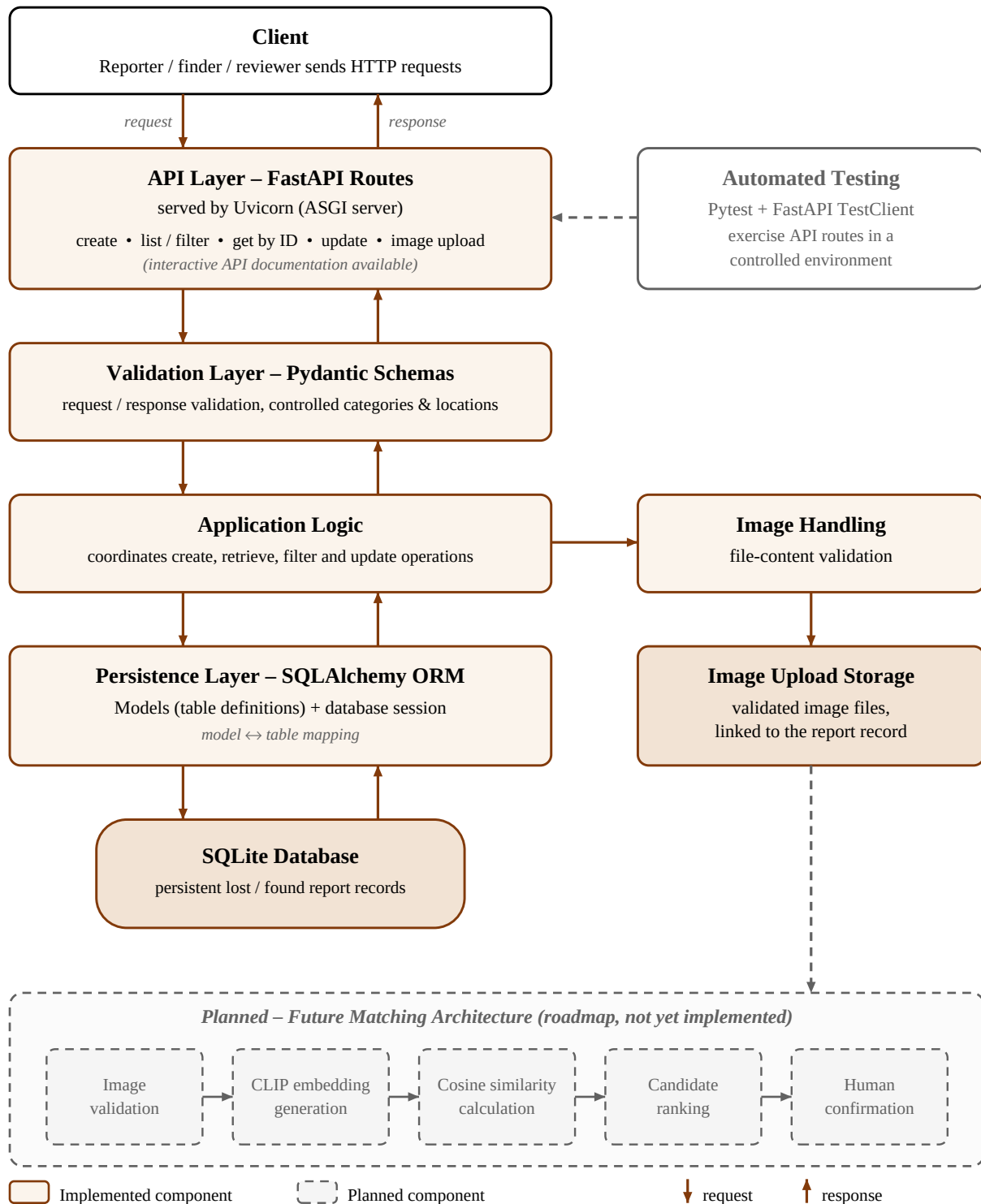


Figure 4.1: High-Level Architecture of the Lost and Found Item Tracker (Eye of Odin)

Figure 4.1 shows how a request travels through the system. The client calls a FastAPI route, Pydantic validates the data, application logic coordinates the operation, and SQLAlchemy stores or retrieves the record in SQLite. Uploaded images pass through file-content validation before storage. The dashed band shows the documented future matching path, a roadmap item rather than a current feature.

Evidence note for this page: retain a concrete project artifact that demonstrates how the selected technologies fit together into a coherent, replaceable, and understandable stack. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

4.2 Architecture Layers

The client submits requests; FastAPI routes them; Pydantic validates structured data; application logic coordinates the operation; SQLAlchemy communicates with SQLite; and image handling validates uploaded files.

The result is a predictable flow from request to persistent record.

A representative campus situation helps explain layer separation between API routes, schemas, models, persistence, and supporting utilities. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For layer separation between API routes, schemas, models, persistence, and supporting utilities, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around layer separation between API routes, schemas, models, persistence, and supporting utilities. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is layer separation between API routes, schemas, models, persistence, and supporting utilities. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, layer separation between API routes, schemas, models, persistence, and supporting utilities. The repository separates responsibilities so that a change in one part does not automatically

require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates the system architecture from client requests through routes, validation, database operations, and responses. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

4.3 Data Flow

A typical create operation begins with a user submitting a lost or found report. The request is validated, converted into a database model, stored, and returned as a structured response.

Retrieval follows the reverse path: the API queries persistence and returns the stored record.

The engineering evidence for this area should be observable. For the lifecycle of a report from request submission through validation, storage, retrieval, and update, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the lifecycle of a report from request submission through validation, storage, retrieval, and update. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the lifecycle of a report from request submission through validation, storage, retrieval, and update. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the lifecycle of a report from request submission through validation, storage, retrieval, and update. The repository separates responsibilities so that a change in one part does not

automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the lifecycle of a report from request submission through validation, storage, retrieval, and update. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates layer separation between API routes, schemas, models, persistence, and supporting utilities. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

4.4 Future Matching Architecture

A future visual matching flow can be added after image storage: image validation, CLIP embedding generation, cosine similarity calculation, candidate ranking, and human confirmation.

This keeps automated similarity as a recommendation mechanism rather than treating a numerical similarity value as proof of ownership.

There is also an important scope boundary around a future matching path in which image representations become candidates for similarity-based discovery. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is a future matching path in which image representations become candidates for similarity-based discovery. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, a future matching path in which image representations become candidates for similarity-based discovery. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated

before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain a future matching path in which image representations become candidates for similarity-based discovery. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For a future matching path in which image representations become candidates for similarity-based discovery, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates the lifecycle of a report from request submission through validation, storage, retrieval, and update. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -5 DESIGN

5.1 Design and Database Schema

The central point of this section is design principles for clarity, consistent fields, controlled values, predictable routes, and maintainable modules. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, design principles for clarity, consistent fields, controlled values, predictable routes, and maintainable modules. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain design principles for clarity, consistent fields, controlled values, predictable routes, and maintainable modules. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For design principles for clarity, consistent fields, controlled values, predictable routes, and maintainable modules, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around design principles for clarity, consistent fields, controlled values, predictable routes, and maintainable modules. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates a future matching path in which image representations become candidates for similarity-based discovery. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -6 IMPLEMENTATION

6.1 Implementation Overview

From an implementation perspective, the implementation strategy and how the repository can be understood as cooperating backend components. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the implementation strategy and how the repository can be understood as cooperating backend components. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the implementation strategy and how the repository can be understood as cooperating backend components, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the implementation strategy and how the repository can be understood as cooperating backend components. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the implementation strategy and how the repository can be understood as cooperating backend components. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates design principles for clarity, consistent fields, controlled values, predictable routes, and maintainable modules. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

6.2 Application Entry Point

The main application initializes the FastAPI service and exposes the routes used by clients. The entry point is intentionally small so that database and validation details remain in their respective modules.

A clear entry point simplifies local execution and future deployment.

A representative campus situation helps explain application startup, FastAPI initialization, route registration, and the role of the entry point. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For application startup, FastAPI initialization, route registration, and the role of the entry point, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around application startup, FastAPI initialization, route registration, and the role of the entry point. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is application startup, FastAPI initialization, route registration, and the role of the entry point. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, application startup, FastAPI initialization, route registration, and the role of the entry point. The repository separates responsibilities so that a change in one part does not automatically

require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates the implementation strategy and how the repository can be understood as cooperating backend components. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

6.3 Database Module

The database module is responsible for SQLite connectivity and session configuration. SQLAlchemy sessions provide the bridge between endpoint logic and stored records.

Keeping connection management centralized avoids duplicating database setup across routes.

The engineering evidence for this area should be observable. For database configuration, session management, table creation, and persistence responsibilities, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around database configuration, session management, table creation, and persistence responsibilities. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is database configuration, session management, table creation, and persistence responsibilities. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, database configuration, session management, table creation, and persistence responsibilities. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated

before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain database configuration, session management, table creation, and persistence responsibilities. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates application startup, FastAPI initialization, route registration, and the role of the entry point. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

6.4 Model Layer

SQLAlchemy models represent persistent report information. The model layer defines how application objects correspond to database records.

The model should remain focused on persistence rather than user-interface concerns.

There is also an important scope boundary around the report model and how stored fields represent the information required by the application. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the report model and how stored fields represent the information required by the application. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the report model and how stored fields represent the information required by the application. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being

hidden inside one large procedure.

A representative campus situation helps explain the report model and how stored fields represent the information required by the application. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the report model and how stored fields represent the information required by the application, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates database configuration, session management, table creation, and persistence responsibilities. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

6.5 Schema Layer

Pydantic schemas define validated request and response structures. This creates a contract between the client and API.

Schemas are useful for preventing invalid values and for documenting what an endpoint expects.

The central point of this section is request and response schemas, validation rules, serialization, and the boundary between data and storage. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, request and response schemas, validation rules, serialization, and the boundary between data and storage. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain request and response schemas, validation rules, serialization, and the boundary between data and storage. A student may notice an item is missing after leaving a classroom,

while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For request and response schemas, validation rules, serialization, and the boundary between data and storage, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around request and response schemas, validation rules, serialization, and the boundary between data and storage. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates the report model and how stored fields represent the information required by the application. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

6.6 Report Creation

A create request contains the information required to register a lost or found item. The API validates the request and persists the resulting record.

A successful response can provide the new record identifier, allowing later operations to refer to the same report.

From an implementation perspective, creating a report, validating its fields, persisting it, and returning a stable record to the caller. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain creating a report, validating its fields, persisting it, and returning a stable record to the caller. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location,

description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For creating a report, validating its fields, persisting it, and returning a stable record to the caller, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around creating a report, validating its fields, persisting it, and returning a stable record to the caller. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is creating a report, validating its fields, persisting it, and returning a stable record to the caller. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates request and response schemas, validation rules, serialization, and the boundary between data and storage. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

6.7 Retrieval, Listing and Filtering

The report API supports collection retrieval and individual record retrieval. Filtering provides a way to narrow the result set using supported attributes.

A not-found response is appropriate when a requested identifier does not exist.

A representative campus situation helps explain listing reports, retrieving individual records, and applying filters to reduce irrelevant results. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For listing reports, retrieving individual records, and applying filters to reduce irrelevant results, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around listing reports, retrieving individual records, and applying filters to reduce irrelevant results. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is listing reports, retrieving individual records, and applying filters to reduce irrelevant results. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, listing reports, retrieving individual records, and applying filters to reduce irrelevant results. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates creating a report, validating its fields, persisting it, and returning a stable record to the caller. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

6.8 Update Operations

Updating a report allows information to change as the item lifecycle progresses. For example, status or descriptive information may be corrected.

The same validation principles should apply to updates so that the database remains consistent.

The engineering evidence for this area should be observable. For updating an existing report while preserving its identity and maintaining valid stored data, a reviewer can examine the request sent to the application, the

validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around updating an existing report while preserving its identity and maintaining valid stored data. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is updating an existing report while preserving its identity and maintaining valid stored data. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, updating an existing report while preserving its identity and maintaining valid stored data. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain updating an existing report while preserving its identity and maintaining valid stored data. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates listing reports, retrieving individual records, and applying filters to reduce irrelevant results. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

6.9 Image Upload

The image endpoint handles user-supplied image data. File content is checked rather than relying only on the filename extension.

This provides a more defensive upload process and prepares the system for future image analysis.

There is also an important scope boundary around image uploads, file validation, storage decisions, and the relationship between images and reports. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is image uploads, file validation, storage decisions, and the relationship between images and reports. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, image uploads, file validation, storage decisions, and the relationship between images and reports. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain image uploads, file validation, storage decisions, and the relationship between images and reports. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For image uploads, file validation, storage decisions, and the relationship between images and reports, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates updating an existing report while preserving its identity and maintaining valid stored data. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

6.10 Testing and Implementation Quality

Automated tests provide a repeatable way to verify the API. The implementation should be checked for valid creation, retrieval, update, filtering, missing records, invalid input, and image validation.

The overall design prioritizes clear module boundaries and future extensibility.

The central point of this section is implementation quality, testability, separation of concerns, and disciplined development practices. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, implementation quality, testability, separation of concerns, and disciplined development practices. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain implementation quality, testability, separation of concerns, and disciplined development practices. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For implementation quality, testability, separation of concerns, and disciplined development practices, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around implementation quality, testability, separation of concerns, and disciplined development practices. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates image uploads, file validation, storage decisions, and the relationship between images and reports. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -7 FEATURES

7.1 Lost Item Reporting

From an implementation perspective, the feature set as a complete reporting workflow rather than a collection of isolated endpoints. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the feature set as a complete reporting workflow rather than a collection of isolated endpoints. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the feature set as a complete reporting workflow rather than a collection of isolated endpoints, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the feature set as a complete reporting workflow rather than a collection of isolated endpoints. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the feature set as a complete reporting workflow rather than a collection of isolated endpoints. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates implementation quality, testability, separation of concerns, and disciplined development practices. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

7.2 Found Item Reporting

A found-item report allows a person to record an item that has been recovered. The same structured approach helps potential owners search the available records.

The report can include descriptive information and an image where applicable.

A representative campus situation helps explain the found-item workflow, including category, location, description, and evidence supplied by a finder. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the found-item workflow, including category, location, description, and evidence supplied by a finder, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the found-item workflow, including category, location, description, and evidence supplied by a finder. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the found-item workflow, including category, location, description, and evidence supplied by a finder. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the found-item workflow, including category, location, description, and evidence supplied by a finder. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates the feature set as a complete reporting workflow rather than a collection of isolated endpoints. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

7.3 Report Search and Filtering

Filtering is important because a campus can accumulate many reports. Category and location provide meaningful dimensions for narrowing results.

A future frontend can expose these filters as forms, dropdowns, or search controls.

The engineering evidence for this area should be observable. For search and filtering behavior that helps users narrow a larger collection of reports, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around search and filtering behavior that helps users narrow a larger collection of reports. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is search and filtering behavior that helps users narrow a larger collection of reports. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, search and filtering behavior that helps users narrow a larger collection of reports. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence,

and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain search and filtering behavior that helps users narrow a larger collection of reports. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates the found-item workflow, including category, location, description, and evidence supplied by a finder. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

7.4 Report Identification

Each report is associated with a unique identifier. This identifier supports direct retrieval and update operations.

Stable identifiers also help administrative workflows and future notification systems refer to a particular report.

There is also an important scope boundary around stable report identifiers and why traceable records matter for later retrieval and updates. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is stable report identifiers and why traceable records matter for later retrieval and updates. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, stable report identifiers and why traceable records matter for later retrieval and updates. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain stable report identifiers and why traceable records matter for later retrieval and updates. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For stable report identifiers and why traceable records matter for later retrieval and updates, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates search and filtering behavior that helps users narrow a larger collection of reports. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

7.5 Image Support

Images provide additional information that may not be captured fully by text. The current system validates image content before handling it.

The stored images can become the input for future computer-vision matching.

The central point of this section is image support as an evidence channel and as preparation for future visual matching. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, image support as an evidence channel and as preparation for future visual matching. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain image support as an evidence channel and as preparation for future visual matching. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location,

description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For image support as an evidence channel and as preparation for future visual matching, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around image support as an evidence channel and as preparation for future visual matching. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates stable report identifiers and why traceable records matter for later retrieval and updates. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

7.6 Validation and Data Quality

Validation ensures that structured fields conform to expected values. This reduces inconsistent records and makes later filtering more reliable.

Data quality is especially important if similarity or recommendation algorithms are introduced.

From an implementation perspective, validation and controlled vocabularies as mechanisms for improving data quality. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain validation and controlled vocabularies as mechanisms for improving data quality. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For validation and controlled vocabularies as mechanisms for improving data quality, a reviewer can examine the request sent to the application, the

validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around validation and controlled vocabularies as mechanisms for improving data quality. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is validation and controlled vocabularies as mechanisms for improving data quality. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates image support as an evidence channel and as preparation for future visual matching. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

7.7 Administrative Extension

A future administrative interface can consume the same API to review, update, and manage reports. Authentication and authorization should be introduced before sensitive administrative actions are exposed.

The current architecture provides the API foundation for such an interface.

A representative campus situation helps explain administrative extensions such as review queues, status handling, audit views, and controlled access. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For administrative extensions such as review queues, status handling, audit views, and controlled access, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around administrative extensions such as review queues, status handling, audit views, and controlled access. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is administrative extensions such as review queues, status handling, audit views, and controlled access. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, administrative extensions such as review queues, status handling, audit views, and controlled access. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates validation and controlled vocabularies as mechanisms for improving data quality. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

7.8 Eye of Odin Concept

Eye of Odin represents the planned progression from basic lost-and-found record management toward intelligent candidate discovery. The vision is to combine structured report data with visual similarity.

Human confirmation should remain part of the resolution process when automated similarity is introduced.

The engineering evidence for this area should be observable. For the Eye of Odin project identity and how the concept maps to observation, organization, and discovery, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the Eye of Odin project identity and how the concept maps to observation, organization, and discovery. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the Eye of Odin project identity and how the concept maps to observation, organization, and discovery. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the Eye of Odin project identity and how the concept maps to observation, organization, and discovery. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the Eye of Odin project identity and how the concept maps to observation, organization, and discovery. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates administrative extensions such as review queues, status handling, audit views, and controlled access. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -8 TESTING

8.1 Testing Strategy

There is also an important scope boundary around the testing strategy and how requirements are translated into repeatable checks. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the testing strategy and how requirements are translated into repeatable checks. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the testing strategy and how requirements are translated into repeatable checks. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the testing strategy and how requirements are translated into repeatable checks. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the testing strategy and how requirements are translated into repeatable checks, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates the Eye of Odin project identity and how the concept maps to observation, organization, and discovery. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot,

depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

8.2 Create Report Tests

Valid lost and found payloads should produce successful creation responses. Tests should verify that required values are stored and returned correctly.

Invalid values should result in validation errors rather than partial records.

The central point of this section is creation tests for valid input, invalid input, persistence, and response structure. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, creation tests for valid input, invalid input, persistence, and response structure. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain creation tests for valid input, invalid input, persistence, and response structure. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For creation tests for valid input, invalid input, persistence, and response structure, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around creation tests for valid input, invalid input, persistence, and response structure. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates the testing strategy and how requirements are translated into repeatable checks. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

8.3 Retrieval Tests

Collection retrieval should return stored reports. Individual retrieval should return the expected report when the identifier exists.

An unknown identifier should result in a predictable not-found response.

From an implementation perspective, retrieval tests for single records, collections, missing identifiers, and response consistency. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain retrieval tests for single records, collections, missing identifiers, and response consistency. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For retrieval tests for single records, collections, missing identifiers, and response consistency, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around retrieval tests for single records, collections, missing identifiers, and response consistency. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is retrieval tests for single records, collections, missing identifiers, and response consistency. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats

each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates creation tests for valid input, invalid input, persistence, and response structure. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

8.4 Filtering Tests

Filtering should be tested using valid category, location, or other supported fields. Results should correspond to the selected filter.

Tests should also cover filters that return no matches.

A representative campus situation helps explain filtering tests for category, location, status, and combinations of query parameters. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For filtering tests for category, location, status, and combinations of query parameters, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around filtering tests for category, location, status, and combinations of query parameters. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is filtering tests for category, location, status, and combinations of query parameters. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, filtering tests for category, location, status, and combinations of query parameters. The repository separates responsibilities so that a change in one part does not automatically require

rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates retrieval tests for single records, collections, missing identifiers, and response consistency. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

8.5 Update Tests

An existing report should accept a valid update and return the updated state. An invalid update should be rejected by validation.

The test should verify that unrelated fields are not unintentionally changed.

The engineering evidence for this area should be observable. For update tests covering successful modification, invalid values, and non-existent records, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around update tests covering successful modification, invalid values, and non-existent records. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is update tests covering successful modification, invalid values, and non-existent records. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, update tests covering successful modification, invalid values, and non-existent records. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being

hidden inside one large procedure.

A representative campus situation helps explain update tests covering successful modification, invalid values, and non-existent records. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates filtering tests for category, location, status, and combinations of query parameters. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

8.6 Image Tests

Valid image content should be accepted by the image endpoint. Non-image content should be rejected when the validation rules require an image.

This test area is important because uploaded files are external input.

There is also an important scope boundary around image tests covering accepted content, invalid files, missing uploads, and association with reports. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is image tests covering accepted content, invalid files, missing uploads, and association with reports. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, image tests covering accepted content, invalid files, missing uploads, and association with reports. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain image tests covering accepted content, invalid files, missing uploads, and association with reports. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For image tests covering accepted content, invalid files, missing uploads, and association with reports, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates update tests covering successful modification, invalid values, and non-existent records. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

8.7 Integration Testing

Integration tests verify the interaction among routes, schemas, models, and the database. A create-then-retrieve scenario checks the full persistence path.

Such tests provide confidence that individual components work together.

The central point of this section is integration testing across routes, validation, database operations, and stored results. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, integration testing across routes, validation, database operations, and stored results. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain integration testing across routes, validation, database operations, and stored results. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location,

description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For integration testing across routes, validation, database operations, and stored results, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around integration testing across routes, validation, database operations, and stored results. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates image tests covering accepted content, invalid files, missing uploads, and association with reports. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

8.8 Test Cases and Expected Results

The test plan covers creation, retrieval, filtering, updating, missing identifiers, invalid category/location values, and image validation.

The same cases can later be expanded for authentication, notifications, and visual matching.

From an implementation perspective, representative test cases with inputs, expected outcomes, and the reason each case matters. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain representative test cases with inputs, expected outcomes, and the reason each case matters. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For representative test cases with inputs, expected outcomes, and the reason each case matters, a reviewer can examine the request sent to the application, the

validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around representative test cases with inputs, expected outcomes, and the reason each case matters. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is representative test cases with inputs, expected outcomes, and the reason each case matters. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates integration testing across routes, validation, database operations, and stored results. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

8.9 Testing Tools and Reporting

Pytest and FastAPI TestClient provide repeatable automated checks. Test results should be retained as evidence for project evaluation and regression testing.

When the frontend is added, end-to-end browser tests can extend the current API-focused strategy.

A representative campus situation helps explain pytest, TestClient, repeatability, failure diagnosis, and reporting of test evidence. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For pytest, TestClient, repeatability, failure diagnosis, and reporting of test evidence, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around pytest, TestClient, repeatability, failure diagnosis, and reporting of test evidence. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is pytest, TestClient, repeatability, failure diagnosis, and reporting of test evidence. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, pytest, TestClient, repeatability, failure diagnosis, and reporting of test evidence. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates representative test cases with inputs, expected outcomes, and the reason each case matters. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -9 DEPLOYMENT

9.1 Local Deployment

The engineering evidence for this area should be observable. For deployment as a controlled sequence from source checkout through environment creation and service verification, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around deployment as a controlled sequence from source checkout through environment creation and service verification. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is deployment as a controlled sequence from source checkout through environment creation and service verification. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, deployment as a controlled sequence from source checkout through environment creation and service verification. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain deployment as a controlled sequence from source checkout through environment creation and service verification. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates pytest, TestClient, repeatability, failure diagnosis, and reporting of test evidence. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

9.2 Environment Setup

A clean Python environment should be prepared before installing project dependencies. Keeping the environment isolated reduces conflicts between packages.

The repository should remain the source of truth for the project code and configuration structure.

There is also an important scope boundary around environment setup for Python, dependencies, database files, uploads, and local execution. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is environment setup for Python, dependencies, database files, uploads, and local execution. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, environment setup for Python, dependencies, database files, uploads, and local execution. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain environment setup for Python, dependencies, database files, uploads, and local execution. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For environment setup for Python, dependencies, database files, uploads, and local execution, a reviewer can examine the request sent to the application, the

validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates deployment as a controlled sequence from source checkout through environment creation and service verification. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

9.3 Configuration

Deployment-specific settings should be kept outside source code when possible. Secrets should not be committed to a public repository.

Upload directories and database locations should also be configured deliberately.

The central point of this section is configuration management for database paths, upload locations, runtime options, and environment-specific values. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, configuration management for database paths, upload locations, runtime options, and environment-specific values. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain configuration management for database paths, upload locations, runtime options, and environment-specific values. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For configuration management for database paths, upload locations, runtime options, and environment-specific values, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which

functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around configuration management for database paths, upload locations, runtime options, and environment-specific values. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates environment setup for Python, dependencies, database files, uploads, and local execution. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

9.4 API Verification

After starting the service, the available API documentation can be used to verify endpoint availability and request/response structures.

Basic health and create/retrieve tests should be performed before considering a deployment ready.

From an implementation perspective, API verification using the interactive documentation and direct endpoint requests. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain API verification using the interactive documentation and direct endpoint requests. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For API verification using the interactive documentation and direct endpoint requests, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around API verification using the interactive documentation and direct endpoint requests. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is API verification using the interactive documentation and direct endpoint requests. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates configuration management for database paths, upload locations, runtime options, and environment-specific values. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

9.5 Production Considerations

A production deployment should add HTTPS, authentication, authorization, logging, monitoring, backups, and appropriate database infrastructure.

SQLite may remain suitable for a small prototype but a larger multi-user environment may require a server-based database.

A representative campus situation helps explain production-oriented concerns such as process management, database choice, storage, logging, and monitoring. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For production-oriented concerns such as process management, database choice, storage, logging, and monitoring, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around production-oriented concerns such as process management, database choice, storage, logging, and monitoring. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is production-oriented concerns such as process management, database choice, storage, logging, and monitoring. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, production-oriented concerns such as process management, database choice, storage, logging, and monitoring. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates API verification using the interactive documentation and direct endpoint requests. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

9.6 Image Storage Considerations

Local image storage is convenient during development. For larger deployments, object storage can provide better scalability and durability.

The application should retain a secure reference to stored files rather than exposing uncontrolled filesystem access.

The engineering evidence for this area should be observable. For image storage choices, naming, retention, access control, and growth considerations, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around image storage choices, naming, retention, access control, and growth considerations. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is image storage choices, naming, retention, access control, and growth considerations. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, image storage choices, naming, retention, access control, and growth considerations. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain image storage choices, naming, retention, access control, and growth considerations. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates production-oriented concerns such as process management, database choice, storage, logging, and monitoring. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

9.7 Security Considerations

Security measures should include input validation, safe file handling, secret management, access control, and encrypted transport.

Administrative actions should be protected by authentication and authorization before deployment.

There is also an important scope boundary around security considerations around validation, file handling, access boundaries, and least-privilege thinking. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is security considerations around validation, file handling, access boundaries, and least-privilege thinking. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, security considerations around validation, file handling, access boundaries, and least-privilege thinking. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain security considerations around validation, file handling, access boundaries, and least-privilege thinking. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For security considerations around validation, file handling, access boundaries, and least-privilege thinking, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates image storage choices, naming, retention, access control, and growth considerations. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

9.8 Backup and Recovery

Database and uploaded image backups are important because the system's value depends on retaining reports and associated evidence.

Recovery procedures should be tested rather than assuming that backups are automatically usable.

The central point of this section is backup and recovery of the database and uploaded evidence so that reports are not lost unexpectedly. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, backup and recovery of the database and uploaded evidence so that reports are not lost unexpectedly. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain backup and recovery of the database and uploaded evidence so that reports are not lost unexpectedly. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For backup and recovery of the database and uploaded evidence so that reports are not lost unexpectedly, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around backup and recovery of the database and uploaded evidence so that reports are not lost unexpectedly. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates security considerations around validation, file handling, access boundaries, and least-privilege thinking. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

9.9 Deployment Workflow

A practical deployment workflow is: prepare environment, install dependencies, run tests, configure database and storage, start the service, verify endpoints, enable security controls, and monitor the application.

This sequence reduces the risk of deploying an untested or misconfigured service.

From an implementation perspective, a repeatable deployment workflow with checkpoints from code readiness to post-deployment verification. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain a repeatable deployment workflow with checkpoints from code readiness to post-deployment verification. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For a repeatable deployment workflow with checkpoints from code readiness to post-deployment verification, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around a repeatable deployment workflow with checkpoints from code readiness to post-deployment verification. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is a repeatable deployment workflow with checkpoints from code readiness to post- deployment verification. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates backup and recovery of the database and uploaded evidence so that reports are not lost unexpectedly. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Challenge – Fragmented Reporting

A representative campus situation helps explain the major engineering challenges encountered when converting a simple idea into a structured application. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the major engineering challenges encountered when converting a simple idea into a structured application, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the major engineering challenges encountered when converting a simple idea into a structured application. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the major engineering challenges encountered when converting a simple idea into a structured application. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the major engineering challenges encountered when converting a simple idea into a structured application. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates a repeatable deployment workflow with checkpoints from code readiness to post-deployment verification. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

10.2 Challenge – Inconsistent Terminology

Users may describe the same item or location differently. Controlled categories and locations reduce this problem.

The approach also improves the consistency of future filtering.

The engineering evidence for this area should be observable. For terminology differences such as lost, found, claimed, category, and location and how controlled values help, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around terminology differences such as lost, found, claimed, category, and location and how controlled values help. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is terminology differences such as lost, found, claimed, category, and location and how controlled values help. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, terminology differences such as lost, found, claimed, category, and location and how controlled values help. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain terminology differences such as lost, found, claimed, category, and location and how controlled values help. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates the major engineering challenges encountered when converting a simple idea into a structured application. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

10.3 Challenge – User-Uploaded Files

Image uploads create a validation boundary because filenames cannot be trusted as proof of file type.

The project includes content-oriented image validation as a defensive measure.

There is also an important scope boundary around the uncertainty and risk associated with user-supplied image files and content validation. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the uncertainty and risk associated with user-supplied image files and content validation. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the uncertainty and risk associated with user-supplied image files and content validation. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the uncertainty and risk associated with user-supplied image files and content validation. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it

is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the uncertainty and risk associated with user-supplied image files and content validation, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates terminology differences such as lost, found, claimed, category, and location and how controlled values help. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

10.4 Challenge – Matching Ambiguity

Two objects can look similar without being the same physical item. Therefore, visual similarity should be interpreted as a candidate-generation mechanism rather than automatic proof.

Human confirmation remains appropriate.

The central point of this section is the ambiguity of visual and textual matching and why similarity should be treated as candidate generation. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the ambiguity of visual and textual matching and why similarity should be treated as candidate generation. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the ambiguity of visual and textual matching and why similarity should be treated as candidate generation. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the ambiguity of visual and textual matching and why similarity should be treated as candidate generation, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the ambiguity of visual and textual matching and why similarity should be treated as candidate generation. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates the uncertainty and risk associated with user-supplied image files and content validation. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

10.5 Challenge – Prototype Scale

SQLite is convenient for an academic prototype but may not provide the same scaling characteristics as a server-based database in a high-concurrency environment.

A later migration can be supported by the ORM abstraction.

From an implementation perspective, prototype-scale constraints including local SQLite usage, storage growth, concurrent access, and observability. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain prototype-scale constraints including local SQLite usage, storage growth, concurrent access, and observability. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For prototype-scale constraints including local SQLite usage, storage growth, concurrent access, and observability, a reviewer can examine the request sent to

the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around prototype-scale constraints including local SQLite usage, storage growth, concurrent access, and observability. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is prototype-scale constraints including local SQLite usage, storage growth, concurrent access, and observability. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates the ambiguity of visual and textual matching and why similarity should be treated as candidate generation. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

10.6 Challenge – Frontend Dependency

The backend can be tested independently, but a complete campus product requires a user-facing interface.

The API-first approach allows a frontend to be developed separately.

A representative campus situation helps explain the dependency on a future frontend and how the API can remain useful independently. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the dependency on a future frontend and how the API can remain useful independently, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are

implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the dependency on a future frontend and how the API can remain useful independently. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the dependency on a future frontend and how the API can remain useful independently. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the dependency on a future frontend and how the API can remain useful independently. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates prototype-scale constraints including local SQLite usage, storage growth, concurrent access, and observability. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

10.7 Current Limitations

The documented project should not be described as having fully implemented CLIP matching, automated notifications, or a complete production authentication system unless those components are separately completed.

The report therefore distinguishes current foundations from roadmap items.

The engineering evidence for this area should be observable. For current limitations that should be stated honestly instead of presenting planned capabilities as implemented, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented

roadmap.

There is also an important scope boundary around current limitations that should be stated honestly instead of presenting planned capabilities as implemented. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is current limitations that should be stated honestly instead of presenting planned capabilities as implemented. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, current limitations that should be stated honestly instead of presenting planned capabilities as implemented. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain current limitations that should be stated honestly instead of presenting planned capabilities as implemented. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates the dependency on a future frontend and how the API can remain useful independently. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

10.8 Solutions Applied

Modular architecture, structured validation, centralized persistence, and automated tests address many of the immediate development challenges.

These choices also reduce the cost of future enhancement.

There is also an important scope boundary around engineering responses already applied to reduce inconsistency, improve validation, and isolate future modules. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is engineering responses already applied to reduce inconsistency, improve validation, and isolate future modules. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, engineering responses already applied to reduce inconsistency, improve validation, and isolate future modules. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain engineering responses already applied to reduce inconsistency, improve validation, and isolate future modules. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For engineering responses already applied to reduce inconsistency, improve validation, and isolate future modules, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates current limitations that should be stated honestly instead of presenting planned capabilities as implemented. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

10.9 Known Risks and Mitigation

Potential risks include incorrect reports, duplicate records, invalid files, unauthorized changes, and false visual matches. Validation, access control, review workflows, and human confirmation can mitigate these risks.

Future versions can add duplicate detection and audit logs.

The central point of this section is known risks and mitigation strategies covering data quality, uploads, access, scaling, and matching errors. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, known risks and mitigation strategies covering data quality, uploads, access, scaling, and matching errors. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain known risks and mitigation strategies covering data quality, uploads, access, scaling, and matching errors. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For known risks and mitigation strategies covering data quality, uploads, access, scaling, and matching errors, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around known risks and mitigation strategies covering data quality, uploads, access, scaling, and matching errors. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates engineering responses already applied to reduce inconsistency, improve validation, and isolate future modules. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 CLIP-Based Image Representation

From an implementation perspective, the future roadmap and how enhancements can be introduced incrementally without destabilizing the core. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the future roadmap and how enhancements can be introduced incrementally without destabilizing the core. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the future roadmap and how enhancements can be introduced incrementally without destabilizing the core, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the future roadmap and how enhancements can be introduced incrementally without destabilizing the core. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the future roadmap and how enhancements can be introduced incrementally without destabilizing the core. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates known risks and mitigation strategies covering data quality, uploads, access, scaling, and matching errors. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

11.2 Cosine Similarity

Cosine similarity measures the angle between two embedding vectors. A higher similarity indicates that the vectors point in a more similar direction.

The value should be used to rank candidates, not as a standalone ownership decision.

A representative campus situation helps explain cosine similarity as a mathematical comparison for future image embeddings. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For cosine similarity as a mathematical comparison for future image embeddings, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around cosine similarity as a mathematical comparison for future image embeddings. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is cosine similarity as a mathematical comparison for future image embeddings. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, cosine similarity as a mathematical comparison for future image embeddings. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence,

and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates the future roadmap and how enhancements can be introduced incrementally without destabilizing the core. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

11.3 Candidate Matching

A future matching service can compare a new lost-item image with stored found-item images and return the most similar candidates.

The result can be presented to a user or administrator for confirmation.

The engineering evidence for this area should be observable. For candidate matching as a staged process combining retrieval, similarity, filtering, and human confirmation, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around candidate matching as a staged process combining retrieval, similarity, filtering, and human confirmation. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is candidate matching as a staged process combining retrieval, similarity, filtering, and human confirmation. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, candidate matching as a staged process combining retrieval, similarity, filtering, and human confirmation. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated

before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain candidate matching as a staged process combining retrieval, similarity, filtering, and human confirmation. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates cosine similarity as a mathematical comparison for future image embeddings. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

11.4 Notifications

Notification support could alert users when a new found report appears relevant to an existing lost report.

Authentication and ownership of reports would be required before reliable personalized notifications can be implemented.

There is also an important scope boundary around notifications that could inform users when relevant reports change or candidate matches appear. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is notifications that could inform users when relevant reports change or candidate matches appear. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, notifications that could inform users when relevant reports change or candidate matches appear. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being

hidden inside one large procedure.

A representative campus situation helps explain notifications that could inform users when relevant reports change or candidate matches appear. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For notifications that could inform users when relevant reports change or candidate matches appear, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates candidate matching as a staged process combining retrieval, similarity, filtering, and human confirmation. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

11.5 Admin Dashboard and Analytics

An administrative dashboard could display active reports, resolved reports, category distributions, and location trends. Such analytics can help administrators understand recurring lost-and-found patterns.

Any sensitive user information should be protected by appropriate access controls.

The central point of this section is an administrative dashboard for review queues, trends, status summaries, and operational visibility. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, an administrative dashboard for review queues, trends, status summaries, and operational visibility. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain an administrative dashboard for review queues, trends, status summaries, and operational visibility. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For an administrative dashboard for review queues, trends, status summaries, and operational visibility, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around an administrative dashboard for review queues, trends, status summaries, and operational visibility. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates notifications that could inform users when relevant reports change or candidate matches appear. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

11.6 Scalable Architecture

As the project grows, database storage, image storage, API services, and matching workloads can be separated. Background processing can handle embedding generation without blocking the main request path.

This modular evolution is consistent with the project's current architecture.

From an implementation perspective, scalable architecture options including stronger databases, object storage, asynchronous jobs, and service separation. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain scalable architecture options including stronger databases, object storage, asynchronous jobs, and service separation. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first

step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For scalable architecture options including stronger databases, object storage, asynchronous jobs, and service separation, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around scalable architecture options including stronger databases, object storage, asynchronous jobs, and service separation. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is scalable architecture options including stronger databases, object storage, asynchronous jobs, and service separation. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates an administrative dashboard for review queues, trends, status summaries, and operational visibility. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -12 CONCLUSION

12.1 Summary

A representative campus situation helps explain the conclusion structure and how the project addresses its original information-management problem. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the conclusion structure and how the project addresses its original information-management problem, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the conclusion structure and how the project addresses its original information-management problem. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the conclusion structure and how the project addresses its original information-management problem. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the conclusion structure and how the project addresses its original information-management problem. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates scalable architecture options including stronger databases, object storage, asynchronous jobs, and service separation. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

12.2 Achievement – API Foundation

The backend-oriented design establishes resource-based operations for report management. FastAPI provides the interface, while validation and persistence provide the data layer.

This creates a stable contract for future clients.

The engineering evidence for this area should be observable. For the achieved API foundation and the value of separating request handling from persistence logic, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the achieved API foundation and the value of separating request handling from persistence logic. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the achieved API foundation and the value of separating request handling from persistence logic. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the achieved API foundation and the value of separating request handling from persistence logic. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the achieved API foundation and the value of separating request handling from persistence logic. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates the conclusion structure and how the project addresses its original information-management problem. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

12.3 Achievement – Data Management

SQLAlchemy and SQLite provide structured persistence. Pydantic schemas help maintain data quality.

Together these components provide a manageable academic prototype.

There is also an important scope boundary around the achieved data-management approach using structured fields, controlled values, and persistent records. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the achieved data-management approach using structured fields, controlled values, and persistent records. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the achieved data-management approach using structured fields, controlled values, and persistent records. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the achieved data-management approach using structured fields, controlled values, and persistent records. A student may notice an item is missing after leaving a classroom,

while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the achieved data-management approach using structured fields, controlled values, and persistent records, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates the achieved API foundation and the value of separating request handling from persistence logic. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

12.4 Achievement – Image Handling

Image support extends the report beyond plain text. File-content validation creates a safer input boundary.

The stored images also provide a future data source for visual similarity.

The central point of this section is the achieved image-handling foundation and the importance of validating file content before later processing. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the achieved image-handling foundation and the importance of validating file content before later processing. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the achieved image-handling foundation and the importance of validating file content before later processing. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the achieved image-handling foundation and the importance of validating file content before later processing, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the achieved image-handling foundation and the importance of validating file content before later processing. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates the achieved data-management approach using structured fields, controlled values, and persistent records. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

12.5 Achievement – Testing

Pytest and TestClient support automated verification of endpoint behavior. The test strategy covers successful operations and invalid input paths.

This provides a repeatable way to identify regressions.

From an implementation perspective, the achieved testing approach and the role of repeatable automated checks in development quality. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the achieved testing approach and the role of repeatable automated checks in development quality. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the achieved testing approach and the role of repeatable automated checks in development quality, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the achieved testing approach and the role of repeatable automated checks in development quality. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the achieved testing approach and the role of repeatable automated checks in development quality. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates the achieved image-handling foundation and the importance of validating file content before later processing. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

12.6 Skills Learned

The project provides practical experience with REST APIs, FastAPI, schema validation, ORM-based persistence, SQLite, file handling, automated tests, version control, and modular architecture.

It also introduces the design considerations involved in adding computer vision to a conventional web application.

A representative campus situation helps explain skills developed through requirements analysis, API design, data modeling, testing, documentation, and teamwork. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For skills developed through requirements analysis, API design, data modeling, testing, documentation, and teamwork, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around skills developed through requirements analysis, API design, data modeling, testing, documentation, and teamwork. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is skills developed through requirements analysis, API design, data modeling, testing, documentation, and teamwork. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, skills developed through requirements analysis, API design, data modeling, testing, documentation, and teamwork. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates the achieved testing approach and the role of repeatable automated checks in development quality. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

12.7 Academic Significance

The project demonstrates the conversion of a real campus problem into a software engineering solution. Requirements, architecture, implementation, testing, limitations, and future work are connected within a single system.

This makes the project suitable as a practical academic case study.

The engineering evidence for this area should be observable. For the academic significance of connecting classroom concepts with a concrete software system, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the academic significance of connecting classroom concepts with a concrete software system. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the academic significance of connecting classroom concepts with a concrete software system. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the academic significance of connecting classroom concepts with a concrete software system. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the academic significance of connecting classroom concepts with a concrete software system. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates skills developed through requirements analysis, API design, data modeling, testing, documentation, and teamwork. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

12.8 Future Direction

The next major step is to connect a user-facing interface and implement the documented visual matching roadmap. Authentication, notifications, administrative workflows, and scalable storage can follow.

Each enhancement can build on the current API foundation.

There is also an important scope boundary around the future direction from a working reporting foundation toward matching, notifications, administration, and scale. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the future direction from a working reporting foundation toward matching, notifications, administration, and scale. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the future direction from a working reporting foundation toward matching, notifications, administration, and scale. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the future direction from a working reporting foundation toward matching, notifications, administration, and scale. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the future direction from a working reporting foundation toward matching, notifications, administration, and scale, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates the academic significance of connecting classroom concepts with a concrete software system. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

12.9 Conclusion

The project establishes a clear foundation for a campus lost-and-found platform. Its strongest architectural characteristic is separation of responsibilities, which allows future functionality to be introduced incrementally.

The final system can evolve from structured reporting into a broader assisted-recovery platform while retaining human oversight for important decisions.

The central point of this section is the final conclusion linking the problem, implemented foundation, documented limits, and future extension path. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, the final conclusion linking the problem, implemented foundation, documented limits, and future extension path. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the final conclusion linking the problem, implemented foundation, documented limits, and future extension path. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the final conclusion linking the problem, implemented foundation, documented limits, and future extension path, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the final conclusion linking the problem, implemented foundation, documented limits, and future extension path. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates the future direction from a working reporting foundation toward matching, notifications, administration, and scale. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -13 REFERENCES

13.1 References – Project and Framework Documentation

From an implementation perspective, the reference strategy and how official framework documentation and project materials support technical claims. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain the reference strategy and how official framework documentation and project materials support technical claims. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For the reference strategy and how official framework documentation and project materials support technical claims, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around the reference strategy and how official framework documentation and project materials support technical claims. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is the reference strategy and how official framework documentation and project materials support technical claims. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates the final conclusion linking the problem, implemented foundation, documented limits, and future extension path. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

13.2 References – Future Computer Vision and General Tools

The documented future matching roadmap refers to CLIP-style image embeddings and cosine similarity. These references should be updated with the exact implementation documentation if the matching module is implemented during the project.

Python documentation provides language and standard-library references. Git documentation provides version-control references. The GitHub repository remains the primary project-specific source for implementation structure and project evolution.

A representative campus situation helps explain future computer-vision and general technical references that can guide later implementation without overstating current scope. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For future computer-vision and general technical references that can guide later implementation without overstating current scope, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around future computer-vision and general technical references that can guide later implementation without overstating current scope. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is future computer-vision and general technical references that can guide later implementation without overstating current scope. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a

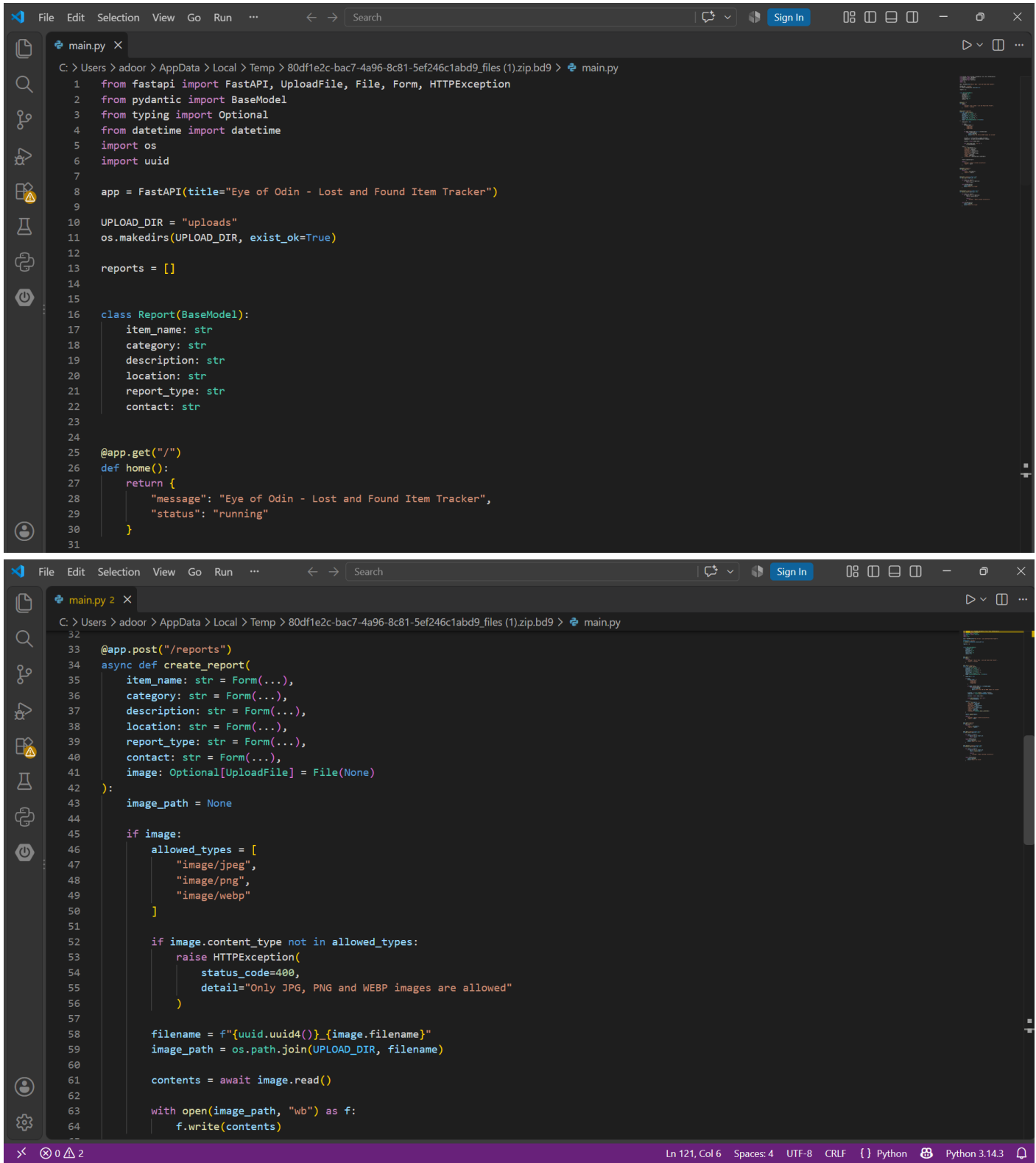
temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, future computer-vision and general technical references that can guide later implementation without overstating current scope. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates the reference strategy and how official framework documentation and project materials support technical claims. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

CHAPTER -14 APPENDICES

SAMPLE CODE SNIPPETS-



```
1 from fastapi import FastAPI, UploadFile, File, Form, HTTPException
2 from pydantic import BaseModel
3 from typing import Optional
4 from datetime import datetime
5 import os
6 import uuid
7
8 app = FastAPI(title="Eye of Odin - Lost and Found Item Tracker")
9
10 UPLOAD_DIR = "uploads"
11 os.makedirs(UPLOAD_DIR, exist_ok=True)
12
13 reports = []
14
15
16 class Report(BaseModel):
17     item_name: str
18     category: str
19     description: str
20     location: str
21     report_type: str
22     contact: str
23
24
25 @app.get("/")
26 def home():
27     return {
28         "message": "Eye of Odin - Lost and Found Item Tracker",
29         "status": "running"
30     }
31
32
33 @app.post("/reports")
34 async def create_report(
35     item_name: str = Form(...),
36     category: str = Form(...),
37     description: str = Form(...),
38     location: str = Form(...),
39     report_type: str = Form(...),
40     contact: str = Form(...),
41     image: Optional[UploadFile] = File(None)
42 ):
43     image_path = None
44
45     if image:
46         allowed_types = [
47             "image/jpeg",
48             "image/png",
49             "image/webp"
50         ]
51
52         if image.content_type not in allowed_types:
53             raise HTTPException(
54                 status_code=400,
55                 detail="Only JPG, PNG and WEBP images are allowed"
56             )
57
58         filename = f"{uuid.uuid4()}_{image.filename}"
59         image_path = os.path.join(UPLOAD_DIR, filename)
60
61         contents = await image.read()
62
63         with open(image_path, "wb") as f:
64             f.write(contents)
```

14.1 Appendix – Project and Team Details

Project: Lost and Found Item Tracker. Project name/concept: Eye of Odin. Project type: Trimester-end. Cluster: 3. Academic year: 2026–2027. Institution: KLH Bachupally. Department: Computer Science and Engineering. Degree: B.Tech.

The same principle applies to testing. Each module can be verified independently, while integration tests can confirm that the modules work together. This provides a practical way to detect errors before a larger user interface is added.

The engineering evidence for this area should be observable. For project and team details that make the report traceable to the academic submission, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around project and team details that make the report traceable to the academic submission. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is project and team details that make the report traceable to the academic submission. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, project and team details that make the report traceable to the academic submission. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain project and team details that make the report traceable to the academic submission. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates future computer-vision and general technical references that can guide later implementation without overstating current scope. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

14.2 Appendix – Guide and Academic Details

Guides: Dr. S. Spandana and Dr. Y. Subbarayudu. Designation: Assistant Professor. Course Coordinator: Ramatenki Sateesh Kumar. HOD: Dr. Venkateswara Rao.

Signature areas can be completed after printing or during the official review process.

There is also an important scope boundary around guide, coordinator, HOD, institution, academic year, cluster, and project-type details. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is guide, coordinator, HOD, institution, academic year, cluster, and project-type details. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, guide, coordinator, HOD, institution, academic year, cluster, and project-type details. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain guide, coordinator, HOD, institution, academic year, cluster, and project-type details. A student may notice an item is missing after leaving a classroom, while another person

may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For guide, coordinator, HOD, institution, academic year, cluster, and project-type details, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

Evidence note for this page: retain a concrete project artifact that demonstrates project and team details that make the report traceable to the academic submission. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

14.3 Appendix – Recommended Project Screenshots

Insert actual screenshots from the developed project before final submission. The following evidence is recommended: project folder structure, FastAPI server running, API documentation, report creation, report listing/filtering, individual report retrieval, update operation, image upload, database records, and automated test output.

Screenshots should be captured from the actual implementation and should not be replaced with fabricated images. Each screenshot should have a figure number and a short caption.

The central point of this section is a disciplined screenshot plan covering interface, API documentation, request forms, responses, and test evidence. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, a disciplined screenshot plan covering interface, API documentation, request forms, responses, and test evidence. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain a disciplined screenshot plan covering interface, API documentation, request forms, responses, and test evidence. A student may notice an item is missing after

leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For a disciplined screenshot plan covering interface, API documentation, request forms, responses, and test evidence, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around a disciplined screenshot plan covering interface, API documentation, request forms, responses, and test evidence. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

Evidence note for this page: retain a concrete project artifact that demonstrates guide, coordinator, HOD, institution, academic year, cluster, and project-type details. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

14.4 Appendix – Suggested API Evidence

The report can include screenshots or copied request/response evidence for the report creation endpoint, report listing endpoint, individual report retrieval endpoint, update endpoint, and image upload endpoint.

For each evidence item, show the request method, endpoint path, relevant request body or parameters, and response status. Sensitive configuration values should be hidden.

From an implementation perspective, API evidence that can be captured from the real implementation without fabricating outputs. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain API evidence that can be captured from the real implementation without fabricating outputs. A student may notice an item is missing after leaving a classroom, while another

person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For API evidence that can be captured from the real implementation without fabricating outputs, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around API evidence that can be captured from the real implementation without fabricating outputs. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is API evidence that can be captured from the real implementation without fabricating outputs. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

Evidence note for this page: retain a concrete project artifact that demonstrates a disciplined screenshot plan covering interface, API documentation, request forms, responses, and test evidence. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

14.5 Appendix – User Guide

1. Open the application or API client. 2. Create a lost or found report. 3. Select a supported category and campus location. 4. Enter a clear description. 5. Upload a valid image when applicable. 6. Submit the report. 7. Search or filter existing reports. 8. Open a report by identifier. 9. Update the report when its state changes.

When the future matching module is implemented, users can review suggested candidates generated from image similarity and confirm an actual match.

A representative campus situation helps explain a user guide describing how a student or staff member can work with the reporting workflow. A student may notice an item is missing after leaving a classroom, while another

person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

The engineering evidence for this area should be observable. For a user guide describing how a student or staff member can work with the reporting workflow, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around a user guide describing how a student or staff member can work with the reporting workflow. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is a user guide describing how a student or staff member can work with the reporting workflow. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, a user guide describing how a student or staff member can work with the reporting workflow. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

Evidence note for this page: retain a concrete project artifact that demonstrates API evidence that can be captured from the real implementation without fabricating outputs. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

14.6 Appendix – Review Forms and Future Matching Pseudocode

Review 1: _____ Date: _____

Review 2: _____ Date: _____

The engineering evidence for this area should be observable. For review forms, evidence checklists, and pseudocode for the future matching pipeline, a reviewer can examine the request sent to the application, the validation response, the stored representation, and the result returned by a later retrieval operation. This creates a traceable chain between requirement and behavior. It also helps the team explain which functions are implemented in the present repository and which capabilities remain part of the documented roadmap.

There is also an important scope boundary around review forms, evidence checklists, and pseudocode for the future matching pipeline. Features that require additional infrastructure are not treated as completed simply because the current architecture can support them in principle. This report therefore distinguishes the working backend foundation from proposed extensions. The distinction improves technical honesty and gives future development a clear starting point: existing modules can remain stable while new matching, notification, administrative, or scaling components are introduced in controlled stages.

The central point of this section is review forms, evidence checklists, and pseudocode for the future matching pipeline. In a campus environment, the usefulness of a software decision depends on whether another person can understand the record later without needing the original reporter beside them. The design therefore treats each item report as structured evidence rather than as a temporary message. Clear fields, predictable values, and stable identifiers make the information reusable across retrieval, review, testing, and future extension.

From an implementation perspective, review forms, evidence checklists, and pseudocode for the future matching pipeline. The repository separates responsibilities so that a change in one part does not automatically require rewriting unrelated parts. Requests are received through defined API operations, validated before persistence, and represented through application models and schemas. This separation is valuable during development because a defect can be isolated to validation, routing, persistence, or file handling instead of being hidden inside one large procedure.

A representative campus situation helps explain review forms, evidence checklists, and pseudocode for the future matching pipeline. A student may notice an item is missing after leaving a classroom, while another person may later record a similar object near a library or cafeteria. The correct first step is not to force a match; it is to preserve both observations accurately. The system can then provide the identifiers, category, location, description, and image evidence needed for a later comparison or human review.

Evidence note for this page: retain a concrete project artifact that demonstrates a user guide describing how a student or staff member can work with the reporting workflow. The artifact may be a real request/response, configuration extract, test case, data record, diagram, or implementation screenshot, depending on the section. It should be captured from the actual project rather than recreated as illustrative evidence.

GEOTAG PHOTO WITH GUIDE

